

基于 PagedAttention 的大语言模型推理高效内存管理

Woosuk Kwon^{1,*} Zhuohan Li^{1,*} Siyuan Zhuang¹ Ying Sheng^{1,2} Lianmin Zheng¹ Cody Hao Yu³
Joseph E. Gonzalez¹ Hao Zhang⁴ Ion Stoica¹

¹UC Berkeley ²Stanford University ³Independent Researcher ⁴UC San Diego

Abstract

大语言模型 (LLM) 的高吞吐量推理服务需要同时批处理足够多的请求。然而，现有系统面临严峻挑战，因为每个请求的键值缓存 (KV 缓存) 内存巨大且动态增长和收缩。当管理效率低下时，这些内存会因碎片化和冗余重复而被大量浪费，从而限制了批处理大小。为解决这一问题，我们提出了 PagedAttention——一种受操作系统中经典虚拟内存和分页技术启发的注意力算法。在此基础上，我们构建了 vLLM，一个 LLM 推理服务系统，实现了 (1) KV 缓存内存接近零浪费，以及 (2) 请求内部和请求之间灵活共享 KV 缓存以进一步降低内存使用。我们的评估表明，与最先进的系统（如 FasterTransformer 和 Orca）相比，vLLM 在相同延迟水平下，将主流 LLM 的吞吐量提升了 2–4 倍。当序列更长、模型更大以及解码算法更复杂时，这一提升更为显著。vLLM 的源代码已公开发布于 <https://github.com/vllm-project/vllm>。

1 引言

以 GPT [5, 37] 和 PaLM [9] 为代表的大语言模型 (LLM) 的涌现，催生了编程助手 [6, 18] 和通用聊天机器人 [19, 35] 等新型应用，这些应用正开始深刻影响我们的工作和日常生活。许多云厂商 [34, 44] 正竞相将这些应用以托管服务的形式对外提供。然而，运行这些应用的成本极为高昂，需要大量硬件加速器（如 GPU）的支持。根据近期估算，处理一条 LLM 请求的成本可达传统关键词查询的 10 倍 [43]。鉴于如此高昂的成本，提升 LLM 推理服务系统的吞吐量——从而降低单次请求成本——正变得日益重要。

LLM 的核心是一个自回归 Transformer 模型 [53]。该模型基于输入（提示词）以及此前已生成的输出 token 序列，逐个生成词（token）。对每个请求，这一昂贵的生成过程会反复执行，直到模型输出终止 token。这种串行生成过程使工作负载呈现内存受限的特性，导致 GPU 计算能力未被充分利用，从而限制了推理服务吞吐量。

* 同等贡献。

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SOSP '23, October 23–26, 2023, Koblenz, Germany

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0229-7/23/10.

<https://doi.org/10.1145/3600006.3613165>

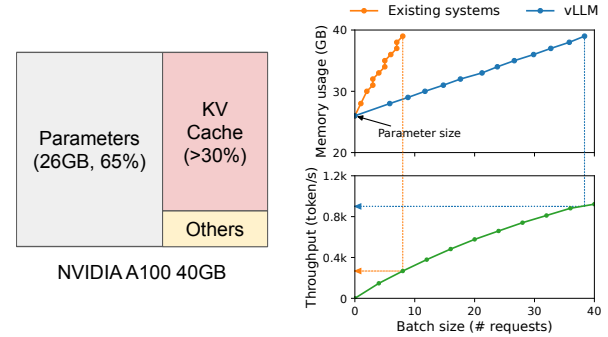


Figure 1. 左：在 NVIDIA A100 上部署一个 130 亿参数 LLM 推理服务时的内存布局。参数（灰色）在推理服务过程中始终驻留在 GPU 内存中。KV 缓存（红色）的内存按每个推理请求进行分配和释放。少量内存（黄色）被临时用于存储激活值。右：vLLM 平滑了现有系统 [31, 60] 中 KV 缓存内存的急剧增长曲线，从而显著提升了推理服务吞吐量。

通过将多个请求批量合并处理，可以提升吞吐量。然而，要在一个批次中处理多个请求，必须高效地管理每个请求的内存空间。例如，图 1（左）展示了在配备 40GB 显存的 NVIDIA A100 GPU 上，一个 130 亿参数 LLM 的内存分布情况。约 65% 的内存被分配给模型权重，这些权重在推理服务过程中保持不变。接近 30% 的内存用于存储请求的动态状态。对于 Transformer 而言，这些状态由注意力机制关联的键（key）和值（value）张量组成，通常被称为 KV 缓存 [41]，它们表示来自先前 token 的上下文，用于按顺序生成新的输出 token。剩余的一小部分内存用于其他数据，包括激活值——即评估 LLM 时产生的临时张量。由于模型权重是恒定的，且激活值仅占用 GPU 内存的一小部分，KV 缓存的管理方式就成为决定最大批量大小的关键因素。当 KV 缓存管理低效时，它会严重限制批量大小，进而制约 LLM 的吞吐量，如图 1（右）所示。

在本文中，我们观察到现有的 LLM 推理服务系统 [31, 60] 未能高效地管理 KV 缓存内存。这主要是因为这些系统将请求的 KV 缓存存储在连续内存空间中，而大多数深度学习框架 [33, 39] 都要求张量存储在连续内存中。然而，与传统深度学习工作负载中的张量不同，KV 缓存具有独特的特性：随着模型生成新 token，它会随时间动态增长和收缩，且其生命周期和长度事先不可知。这些特性使得现有系统的方法在以下两个方面显著低效：

首先，现有系统 [31, 60] 存在内部碎片和外部碎片问题。为了将请求的 KV 缓存存储在连续空间中，它们会按请求的最大长度（例如 2048 个 token）预分配一块连

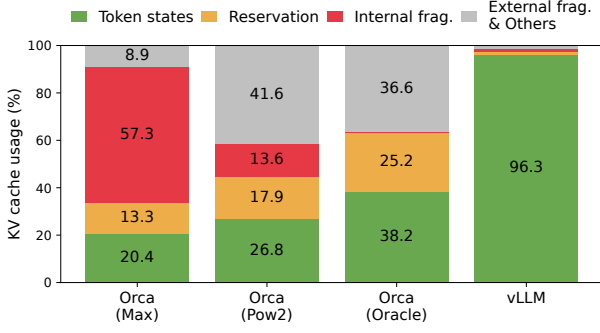


Figure 2. 在 §6.2 的实验中，不同 LLM 推理服务系统的平均内存浪费百分比。

续的内存。这会导致严重的内部碎片，因为请求的实际长度可能远小于其最大长度（例如，图 11）。此外，即使事先知道实际长度，预分配策略仍然低效：由于整块内存存在请求的生命周期内都被预留，其他较短的请求无法利用该块中当前未被使用的任何部分。除此之外，由于每个请求的预分配大小可能不同，外部碎片也可能相当严重。事实上，我们在图 2 中的分析结果显示，在现有系统中，仅有 20.4% 至 38.2% 的 KV 缓存内存被用于存储实际的 token 状态。

其次，现有系统无法利用内存共享的机会。LLM 服务通常使用高级解码算法，如并行采样和束搜索，这些算法会为每个请求生成多个输出。在这些场景中，请求由多个序列组成，这些序列可以部分共享其 KV 缓存。然而，由于各个序列的 KV 缓存存储在独立的连续空间中，现有系统无法实现内存共享。

为了解决上述局限性，我们提出了 *PagedAttention*，这是一种受操作系统（OS）解决内存碎片和共享问题的方案——虚拟内存分页——所启发的注意力算法。*PagedAttention* 将请求的 KV 缓存划分为块（block），每个块可以包含固定数量 token 的注意力键和值。在 *PagedAttention* 中，KV 缓存的各个块不必存储在连续空间中。因此，我们可以像操作系统的虚拟内存一样，以更灵活的方式管理 KV 缓存：可以将块视为页（page），将 token 视为字节（byte），将请求视为进程（process）。这种设计通过使用相对较小的块并按需分配，缓解了内部碎片问题。此外，由于所有块的大小相同，它消除了外部碎片。最后，它支持以块为粒度在同一个请求的不同序列之间，甚至在不同请求之间实现内存共享。

在本工作中，我们构建了 *vLLM*，一个基于 *PagedAttention* 的高吞吐量分布式 LLM 推理服务引擎，它实现了 KV 缓存内存的近乎零浪费。*vLLM* 采用与 *PagedAttention* 协同设计的块级内存管理和抢占式请求调度。*vLLM* 支持各种规模的流行 LLM，如 GPT [5]、OPT [62] 和 LLaMA [52]，包括那些超出单 GPU 内存容量的模型。我们在多种模型和工作负载上的评估表明，与最先进的现有系统 [31, 60] 相比，*vLLM* 将 LLM 推理服务吞吐量提升了 2-4 倍，且完全不影响模型准确率。在处理更长

序列、更大模型和更复杂解码算法 (§4.3) 时，提升效果更加显著。总而言之，本文做出了以下贡献：

- 我们识别了 LLM 推理服务中内存分配面临的挑战，并量化了它们对推理服务性能的影响。
- 我们提出了 *PagedAttention*，一种在非连续分页内存中存储的 KV 缓存上运行的注意力算法，其灵感来源于操作系统中的虚拟内存和分页技术。
- 我们设计并实现了 *vLLM*，一个基于 *PagedAttention* 构建的分布式 LLM 推理服务引擎。
- 我们在多种场景下对 *vLLM* 进行了评估，结果表明它在性能上显著超越了此前最先进的解决方案，如 *FasterTransformer* [31] 和 *Orca* [60]。

2 背景

本节描述典型 LLM 的生成和推理服务流程，以及 LLM 推理服务中使用的迭代级调度机制。

2.1 基于 Transformer 的大语言模型

语言建模的任务是对 token 序列 $(x_1, \dots, x_n)$ 的概率进行建模。由于语言具有自然的顺序性，通常将整个序列上的联合概率分解为条件概率的乘积（即自回归分解 [3]）：

$$P(x) = P(x_1) \cdot P(x_2 | x_1) \cdots P(x_n | x_1, \dots, x_{n-1}). \quad (1)$$

Transformer [53] 已成为大规模建模上述概率的事实标准架构。基于 Transformer 的语言模型最重要的组成部分是其自注意力层。对于输入隐藏状态序列 $(x_1, \dots, x_n) \in \mathbb{R}^{n \times d}$ ，自注意力层首先对每个位置 i 进行线性变换，得到查询（query）、键（key）和值（value）向量：

$$q_i = W_q x_i, k_i = W_k x_i, v_i = W_v x_i. \quad (2)$$

然后，自注意力层通过将一个位置的查询向量与其之前所有键向量相乘来计算注意力分数 a_{ij} ，并将输出 o_i 计算为值向量的加权平均：

$$a_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d})}{\sum_{t=1}^i \exp(q_i^\top k_t / \sqrt{d})}, o_i = \sum_{j=1}^i a_{ij} v_j. \quad (3)$$

除了公式 4 中的计算外，Transformer 模型中的所有其他组件，包括嵌入层、前馈层、层归一化 [2]、残差连接 [22]、输出 logit 计算，以及公式 2 中的查询、键和值变换，均以如下形式独立地应用于每个位置： $y_i = f(x_i)$ 。

2.2 LLM 服务与自回归生成

训练完成后，LLM 通常被部署为条件生成服务（例如，补全 API [34] 或聊天机器人 [19, 35]）。对 LLM 服务的请求提供一组输入提示 token $(x_1, \dots, x_n)$ ，LLM 服务根据公式 1 生成一组输出 token $(x_{n+1}, \dots, x_{n+T})$ 。我们将提示和输出列表的连接称为序列。

由于公式 1 中的分解关系，LLM 只能逐个采样并生成新的 token，且每个新 token 的生成过程依赖于该序列中的所有先前 token，特别是它们的键和值向量。在此顺序生成过程中，现有 token 的键和值向量通常被缓存以供生成未来 token 使用，这被称为 KV 缓存。注意，一个 token 的 KV 缓存依赖于其所有先前 token。这意味着同

一 token 出现在序列中不同位置时，其 KV 缓存将是不同的。

给定一个请求提示，LLM 服务的生成计算可分解为两个阶段：

提示阶段以整个用户提示 $(x_1, \dots, x_n)$ 作为输入，计算第一个新 token 的概率 $P(x_{n+1} | x_1, \dots, x_n)$ 。在此过程中，同时生成键向量 $k_1, \dots, k_n$ 和值向量 $v_1, \dots, v_n$ 。由于提示 token $x_1, \dots, x_n$ 全部已知，提示阶段的计算可通过矩阵-矩阵乘法操作实现并行化。因此，该阶段能高效利用 GPU 固有的并行性。

自回归生成阶段顺序生成剩余的新 token。在第 t 次迭代中，模型以一个 token x_{n+t} 作为输入，利用键向量 $k_1, \dots, k_{n+t}$ 和值向量 $v_1, \dots, v_{n+t}$ 计算概率 $P(x_{n+t+1} | x_1, \dots, x_{n+t})$ 。注意，位置 1 到 $n+t-1$ 的键和值向量已在之前的迭代中缓存，本次迭代仅计算新的键和值向量 k_{n+t} 和 v_{n+t} 。该阶段在序列达到最大长度（由用户指定或受 LLM 限制）或发出序列结束（<eos>）token 时完成。由于数据依赖性，不同迭代的计算无法并行化，通常使用效率较低的矩阵-向量乘法。因此，该阶段严重未充分利用 GPU 计算能力，成为内存受限阶段，占据了单个请求延迟的大部分。

2.3 LLM 的批处理技术

通过批处理多个请求可以提高 LLM 推理服务的计算利用率。由于请求共享相同的模型权重，移动权重的开销在批次中的请求之间分摊，当批处理大小足够大时可被计算开销所淹没。然而，将请求批量送入 LLM 服务并不简单，原因有二。首先，请求可能在不同时间到达。朴素的批处理策略要么让较早的请求等待后续请求，要么将新到达的请求推迟到较早请求完成后才处理，导致显著的排队延迟。其次，请求的输入和输出长度可能差异很大（图 11）。直接的批处理技术会对请求的输入和输出进行填充以统一长度，从而浪费 GPU 计算和内存。

为解决这一问题，细粒度批处理机制已被提出，如 cellular batching [16] 和迭代级调度 [60]。与在请求级别工作的传统方法不同，这些技术在迭代级别运行。每次迭代后，已完成的请求从批次中移除，新的请求被加入。因此，新请求只需等待单次迭代即可被处理，而无需等待整个批次完成。此外，借助专用 GPU kernel，这些技术消除了对输入和输出进行填充的需要。通过减少排队延迟和填充带来的低效问题，细粒度批处理机制显著提升了 LLM 推理服务的吞吐量。

3 LLM 推理服务中的内存挑战

尽管细粒度批处理减少了计算浪费，并使请求能够以更灵活的方式批处理，但能够同时批处理的请求数量仍然受到 GPU 内存容量的限制，特别是分配给存储 KV 缓存的空间。换言之，推理服务系统的吞吐量是内存受限的。克服这一内存瓶颈需要解决内存管理中的以下挑战：

巨大的 KV 缓存。KV 缓存大小随请求数量快速增长。例如，对于 13B 参数的 OPT 模型 [62]，单个 token 的 KV 缓存需要 800 KB 空间，计算方式为 2（键和值向量） $\times$ 5120（隐藏状态大小） $\times$ 40（层数） $\times$ 2（每 FP16 字节数）。由

于 OPT 可生成长达 2048 个 token 的序列，存储一个请求的 KV 缓存所需内存可高达 1.6 GB。当前 GPU 的内存容量在数十 GB 级别。即使将所有可用内存都分配给 KV 缓存，也只能容纳几十个请求。此外，低效的内存管理会进一步减少批处理大小，如图 2 所示。另外，从当前趋势来看，GPU 的计算速度增长快于内存容量增长 [17]。例如，从 NVIDIA A100 到 H100，FLOPS 提升超过 2 倍，但 GPU 内存仍最高停留在 80GB。因此，我们认为内存将成为一个日益显著的瓶颈。

复杂的解码算法。LLM 服务提供一系列解码算法供用户选择，每种算法对内存管理复杂度的要求各不相同。例如，当用户从单个输入提示请求多个随机样本时（程序建议中的典型用例 [18]），提示部分的 KV 缓存可被共享以最小化内存使用，在我们的实验中（§6.3）该部分占总 KV 缓存内存的 12%。另一方面，自回归生成阶段的 KV 缓存应保持不共享，因为不同样本结果各不相同且依赖于上下文和位置。KV 缓存共享的程度取决于所采用的具体解码算法。在更复杂的算法如束搜索 [49] 中，不同请求束可共享更大比例（最多可节省 55% 内存，见 §6.3）的 KV 缓存，且共享模式随解码过程的推进而演变。

未知输入和输出长度的调度。LLM 服务的请求在输入和输出长度上表现出差异性。这要求内存管理系统能够容纳范围广泛的提示长度。此外，随着请求的输出长度在解码过程中增长，其 KV 缓存所需内存也会扩展，并可能耗尽新到达请求或已有提示正在生成中的可用内存。系统需要做出调度决策，例如从 GPU 内存中删除或换出某些请求的 KV 缓存。

3.1 现有系统中的内存管理

由于当前深度学习框架 [33, 39] 中的大多数算子要求张量存储在连续内存中，先前的 LLM 推理服务系统 [31, 60] 也将一个请求的 KV 缓存存储为跨不同位置的连续张量。由于 LLM 的输出长度不可预测，它们根据请求的最大可能序列长度静态分配一块内存，而不考虑请求的实际输入或最终输出长度。

图 3 展示了两个请求：请求 A 的最大可能序列长度为 2048，请求 B 最大为 512。现有系统中的块预分配方案存在三个主要的内存浪费来源：为未来 token 预留的槽位、因过度预分配最大可能序列长度导致的内部碎片，以及来自内存分配器（如伙伴分配器）的外部碎片。外部碎片永远不会用于生成的 token，这在服务请求之前就已知道。内部碎片也始终未使用，但这只有在请求完成采样后才能意识到。两者都是纯内存浪费。虽然预留内存最终会被使用，但在请求的整个持续时间内预留此空间（尤其是当预留空间很大时），占用了本可用于处理其他请求的空间。我们在实验中可视化了内存浪费的平均百分比，如图 2 所示，结果显示在先前系统中，实际有效内存可低至 20.4%。

尽管压缩 [54] 被提出作为解决碎片的潜在方案，但在性能敏感的 LLM 推理服务系统中执行压缩是不切实际的，因为 KV 缓存规模巨大。即使采用压缩，每个请求的预分配块空间仍然阻止了现有内存管理系统中特定于解码算法的内存共享。

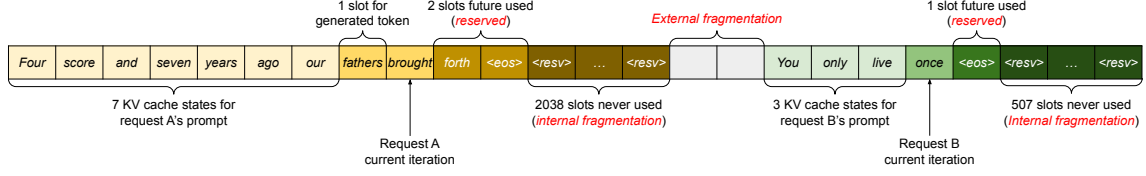


Figure 3. 现有系统中的 KV 缓存内存管理。存在三种类型的内存浪费——预留空间、内部碎片和外部碎片——导致其他请求无法放入内存。每个内存槽位中的 token 代表其 KV 缓存。注意，相同 token 在不同位置可能具有不同的 KV 缓存。

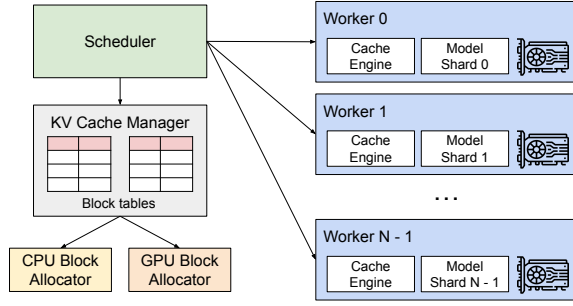


Figure 4. vLLM 系统总览。

4 方法

在本工作中，我们开发了一种新的注意力算法 *PagedAttention*，并构建了一个 LLM 推理服务引擎 *vLLM*，以应对 §3 中概述的挑战。*vLLM* 的架构如图 4 所示。*vLLM* 采用集中式调度器来协调分布式 GPU 工作节点的执行。KV 缓存管理器通过 *PagedAttention* 实现以分页方式高效管理 KV 缓存。具体而言，KV 缓存管理器通过集中式调度器发送的指令来管理 GPU 工作节点上的物理 KV 缓存内存。

接下来，我们将在 §4.1 中描述 *PagedAttention* 算法。在此基础上，分别在 §4.2 和 §4.3 中展示 KV 缓存管理器的设计及其如何支撑 *PagedAttention*。然后，展示该设计如何为各种解码方法提供高效的内存管理 (§4.4) 以及如何处理可变长度的输入和输出序列 (§4.5)。最后，展示 *vLLM* 的系统设计如何在分布式环境下工作 (§4.6)。

4.1 PagedAttention

为解决 §3 中的内存挑战，我们引入了 *PagedAttention*，这是一种受操作系统中经典分页 [25] 思想启发的注意力算法。与传统注意力算法不同，*PagedAttention* 允许将连续的键和值存储在非连续的内存空间中。具体而言，*PagedAttention* 将每个序列的 KV 缓存划分为 KV 块。每个块包含固定数量 token 的键和值向量，¹我们将其记作 KV 块大小 (B) 。记键块 $K_j = (k_{(j-1)B+1}, \dots, k_{jB})$ 和值块 $V_j = (v_{(j-1)B+1}, \dots, v_{jB})$ 。公式 4 中的注意力计算可转化

¹在 Transformer 中，每个 token 在每一层和层内的每个注意力头都有一组键和值向量。所有键和值向量可以一起在单个 KV 块中管理，或者不同头和层的键和值向量可以各自拥有独立的块并在独立的块表中管理。两种设计没有性能差异，我们选择第二种以便于实现。

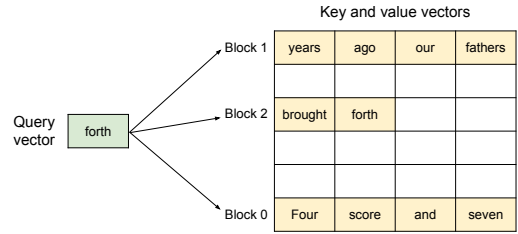


Figure 5. PagedAttention 算法示意图，其中注意力键和值向量以非连续块的形式存储在内存中。

为以下逐块计算形式：

$$A_{ij} = \frac{\exp(q_i^T K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^T K_t / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^T, \quad (4)$$

其中 $A_{ij} = (a_{i,(j-1)B+1}, \dots, a_{i,jB})$ 是第 j 个 KV 块上的注意力分数行向量。

在注意力计算过程中，*PagedAttention* kernel 分别识别并获取不同的 KV 块。我们在图 5 中展示了一个 *PagedAttention* 示例：键和值向量分布在三个块中，且这三个块在物理内存上不连续。每次计算时，kernel 将查询 token (“forth”) 的查询向量 q_i 与一个块中的键向量 K_j (例如，块 0 中 “Four score and seven” 的键向量) 相乘，计算注意力分数 A_{ij} ，然后将 A_{ij} 与该块中的值向量 V_j 相乘，得到最终的注意力输出 o_i 。

总之，*PagedAttention* 算法允许 KV 块存储在非连续的物理内存中，从而在 *vLLM* 中实现了更灵活的分页内存管理。

4.2 KV 缓存管理器

vLLM 内存管理器的核心思想类似于操作系统中的虚拟内存 [25]。操作系统将内存划分为固定大小的页面，并将用户程序的逻辑页面映射到物理页面。连续的逻辑页面可以对应不连续的物理内存页面，使用户程序仿佛在访问连续内存。此外，物理内存空间无需提前完全预留，操作系统可按需动态分配物理页面。*vLLM* 利用虚拟内存背后的思想来管理 LLM 服务中的 KV 缓存。在 *PagedAttention* 的支撑下，我们将 KV 缓存组织为固定大小的 KV 块，如同虚拟内存中的页面。

一个请求的 KV 缓存被表示为一系列逻辑 KV 块，随着新 token 及其 KV 缓存的生成从左到右填充。最后一个 KV 块中未填充的位置预留用于未来的生成。在 GPU 工作节点上，块引擎分配一块连续的 GPU DRAM 并将其划分为物理 KV 块 (CPU RAM 上也同样进行此操作)

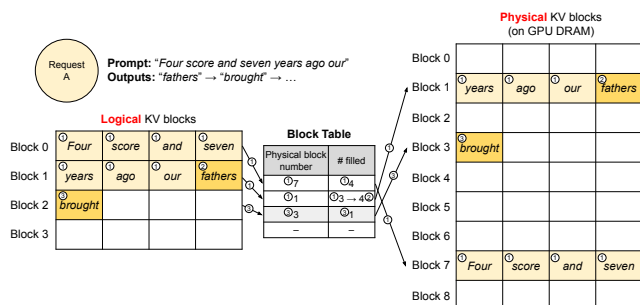


Figure 6. vLLM 中的块表转换。

于换出；见 §4.5)。KV 块管理器还维护块表——即每个请求的逻辑 KV 块与物理 KV 块之间的映射。每个块表条目记录逻辑块对应的物理块以及已填充的位置数。分离逻辑和物理 KV 块使 vLLM 能够动态增长 KV 缓存内存，而无需预先为所有位置预留空间，这消除了现有系统中的大部分内存浪费，如图 2 所示。

4.3 使用 PagedAttention 和 vLLM 进行解码

接下来，我们通过一个示例（如图 6 所示）来演示 vLLM 如何在单个输入序列的解码过程中执行 PagedAttention 并管理内存：① 与操作系统的虚拟内存类似，vLLM 不需要一开始就为最大可能生成序列长度预留内存。相反，它只预留必要的 KV 块来容纳提示计算过程中生成的 KV 缓存。在本例中，提示有 7 个 token，因此 vLLM 将前 2 个逻辑 KV 块 (0 和 1) 映射到 2 个物理 KV 块（分别为 7 和 1）。在预填充步骤中，vLLM 使用传统的自注意力算法（如 [13]）生成提示的 KV 缓存和第一个输出 token。然后，vLLM 将前 4 个 token 的 KV 缓存存储在逻辑块 0 中，将后续 3 个 token 存储在逻辑块 1 中。剩余槽位预留用于后续的回归生成阶段。② 在第一个自回归解码步骤中，vLLM 使用 PagedAttention 算法在物理块 7 和 1 上生成新 token。由于最后一个逻辑块中仍有一个可用槽位，新生成的 KV 缓存存储在此处，并更新块表的 #filled 记录。③ 在第二个解码步骤中，由于最后一个逻辑块已满，vLLM 将新生成的 KV 缓存存储在一个新的逻辑块中；vLLM 为其分配一个新的物理块（物理块 3），并将此映射存储在块表中。

全局来看，对于每次解码迭代，vLLM 首先选择一组候选序列进行批处理（详见 §4.5），并为新需要的逻辑块分配物理块。然后，vLLM 将当前迭代的所有输入 token（即提示阶段请求的所有 token 和生成阶段请求的最新 token）连接为一个序列，送入 LLM。在 LLM 的计算过程中，vLLM 使用 PagedAttention kernel 访问以逻辑 KV 块形式存储的先前 KV 缓存，并将新生成的 KV 缓存保存到物理 KV 块中。在一个 KV 块内存储多个 token（块大小 > 1）使 PagedAttention kernel 能够在更多位置并行处理 KV 缓存，从而提高硬件利用率并降低延迟。然而，较大的块大小也会增加内存碎片。我们在 §7.2 中研究了块大小的影响。

再次强调，vLLM 随着更多 token 及其 KV 缓存的生成，动态地将新的物理块分配给逻辑块。由于所有块从左到右填充，且新物理块仅在所有先前块都填满时才分

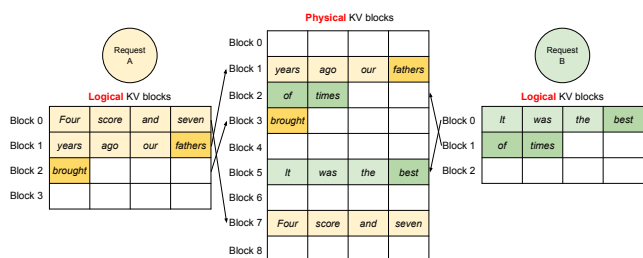


Figure 7. vLLM 中同时存储两个请求的 KV 缓存。

配，vLLM 将一个请求的所有内存浪费限制在一个块内，因此能有效利用所有内存，如图 2 所示。这使得更多请求能够放入内存进行批处理——从而提高吞吐量。一旦请求生成完成，其 KV 块可被释放以存储其他请求的 KV 缓存。在图 7 中，我们展示了 vLLM 管理两个序列内存的示例。两个序列的逻辑块映射到 GPU 工作节点中块引擎预留空间内的不同物理块。两个序列的相邻逻辑块无需在物理 GPU 内存中连续，物理块的空间可被两个序列有效利用。

4.4 在其他解码场景中的应用

§4.3 展示了 PagedAttention 和 vLLM 如何处理基本解码算法，如贪心解码和采样，即以用户提示为输入并生成单个输出序列。在许多成功的 LLM 应用中 [18, 34]，LLM 服务必须提供更复杂的解码场景，这些场景展现出复杂的访问模式和更多的内存共享机会。我们在本节展示 vLLM 对它们的通用适用性。

并行采样。在基于 LLM 的程序助手中 [6, 18]，LLM 为单个输入提示生成多个采样输出；用户可以从多个候选中选择一个偏好的输出。到目前为止，我们隐式地假设一个请求生成单个序列。在本文的其余部分，我们假设更一般的情况，即一个请求生成多个序列。在并行采样中，一个请求包含共享同一输入提示的多个样本，因此提示的 KV 缓存也可以共享。借助其 PagedAttention 和分页内存管理，vLLM 可以轻松实现这种共享并节省内存。

图 8 展示了一个为两个输出进行并行解码的示例。由于两个输出共享相同的提示，我们仅在提示阶段为提示的状态保留一份副本；两个序列提示的逻辑块映射到相同的物理块：两个序列的逻辑块 0 和 1 分别映射到物理块 7 和 1。由于一个物理块可以映射到多个逻辑块，我们为每个物理块引入一个引用计数。在本例中，物理块 7 和 1 的引用计数均为 2。在生成阶段，两个输出采样不同的输出 token，需要为 KV 缓存分别存储。vLLM 对被多个序列修改的物理块实现了块粒度的写时复制机制，类似于操作系统虚拟内存中的写时复制技术（例如，在 fork 进程时）。具体来说，在图 8 中，当样本 A1 需要写入其最后一个逻辑块（逻辑块 1）时，vLLM 识别到对应物理块（物理块 1）的引用计数大于 1；它分配一个新的物理块（物理块 3），指示块引擎从物理块 1 复制信息，并将引用计数减为 1。接下来，当样本 A2 写入物理块 1 时，引用计数已减为 1；因此 A2 直接将新生成的 KV 缓存写入物理块 1。

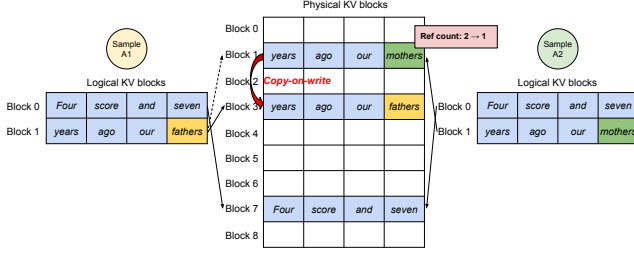


Figure 8. 并行采样示例。

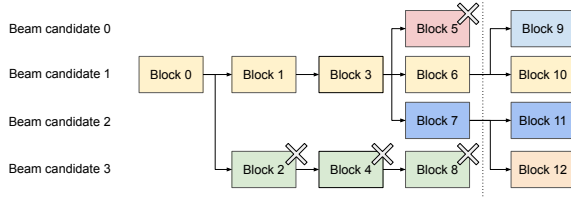


Figure 9. 束搜索示例。

总之，vLLM 支持跨多个输出样本共享用于存储提示 KV 缓存的大部分空间，仅最后一个逻辑块例外，该块通过写时复制机制管理。通过跨多个样本共享物理块，内存使用可大幅降低，特别是对于长输入提示。

束搜索。在机器翻译等 LLM 任务中 [59]，用户期望 LLM 输出前 k 个最合适的翻译结果。束搜索 [49] 被广泛用于从 LLM 中解码最可能的输出序列，因为它降低了完全遍历样本空间的计算复杂度。该算法依赖于束宽度参数 k ，它决定每一步保留的 top 候选序列数量。在解码过程中，束搜索通过考虑所有可能的 token 来扩展束中的每个候选序列，使用 LLM 计算它们各自的概率，并从 $k \cdot |V|$ 个候选中保留前 k 个最可能的序列，其中 $|V|$ 是词汇量大小。

与并行解码不同，束搜索不仅促进初始提示块的共享，还促进不同候选之间其他块的共享，且共享模式随着解码过程的推进动态变化，类似于操作系统中由复合 fork 创建的进程树。图 9 展示了 vLLM 如何管理一个 $k = 4$ 的束搜索示例的 KV 块。在虚线所示的迭代之前，每个候选序列已使用 4 个完整逻辑块。所有束候选共享第一个块 0（即提示）。候选 3 从第二个块开始与其他候选分叉。候选 0-2 共享前 3 个块，并在第四个块处分叉。在后续迭代中，前 4 个最可能的候选全部来自候选 1 和 2。由于原始候选 0 和 3 不再属于 top 候选，它们的逻辑块被释放，对应物理块的引用计数减少。vLLM 释放所有引用计数降至 0 的物理块（块 2、4、5、8）。然后，vLLM 分配新的物理块（块 9-12）来存储来自新候选的新 KV 缓存。此时，所有候选共享块 0、1、3；候选 0 和 1 共享块 6，候选 2 和 3 进一步共享块 7。

先前的 LLM 推理服务系统需要在束候选之间频繁复制 KV 缓存。例如，在图 9 所示的情况下，在虚线之后，候选 3 需要复制候选 2 的大部分 KV 缓存才能继续生成。vLLM 的物理块共享大大降低了这种频繁的内存复制开销。在 vLLM 中，不同束候选的大部分块可以共享。写时复制机制仅在新生成的 token 位于旧的共享块内时应用，如同并行解码中的情况。这仅涉及复制一个数据块。

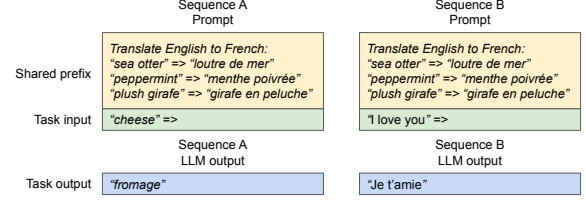


Figure 10. 机器翻译的共享提示示例。示例取自 [5]。

共享前缀。通常情况下，LLM 用户会提供一个（较长的）任务描述，包括指令和示例输入输出，也称为系统提示 [36]。该描述与实际任务输入连接，构成请求的提示。LLM 根据完整提示生成输出。图 10 展示了一个示例。此外，共享前缀可通过提示工程进一步调优，以提高下游任务的准确性 [26, 27]。

对于此类应用，许多用户提示共享一个前缀，因此 LLM 服务提供商可以提前存储该前缀的 KV 缓存，以减少在前缀上的冗余计算。在 vLLM 中，这可以通过 LLM 服务提供商为一组预定义的共享前缀预留一组物理块来方便地实现，类似于操作系统跨进程处理共享库的方式。带有共享前缀的用户输入提示只需将其逻辑块映射到已缓存的物理块（最后一个块标记为写时复制）。提示阶段的计算仅需在用户的任务输入上执行。

混合解码方法。前面讨论的解码方法展现出多样的内存共享和访问模式。尽管如此，vLLM 支持同时处理具有不同解码偏好的请求，而现有系统无法高效地做到这一点。这是因为 vLLM 通过一个通用的映射层将逻辑块转换为物理块，对 LLM 及其执行 kernel 隐藏了不同序列之间的复杂内存共享关系。LLM 及其执行 kernel 只看到每个序列的物理块 ID 列表，无需处理跨序列的共享模式。与现有系统相比，这种方法拓宽了对具有不同采样需求的请求的批处理机会，最终提高了系统的整体吞吐量。

4.5 调度与抢占

当请求流量超过系统容量时，vLLM 必须对请求子集进行优先级排序。在 vLLM 中，我们采用先到先服务 (FCFS) 调度策略来处理所有请求，确保公平性并防止饥饿。当 vLLM 需要抢占请求时，它确保最早到达的请求优先获得服务，最晚到达的请求优先被抢占。

LLM 服务面临一个独特的挑战：LLM 的输入提示长度可能差异显著，且最终的输出长度事先不可知，这取决于输入提示和模型。随着请求数量及其输出的增长，vLLM 可能耗尽 GPU 的物理块来存储新生成的 KV 缓存。在此背景下，vLLM 需要回答两个经典问题：(1) 应该逐出哪些块？(2) 如果需要再次使用，如何恢复已被逐出的块？通常，逐出策略使用启发式方法预测哪个块将在最远的未来被访问，并将该块逐出。由于在我们的情况下我们知道一个序列的所有块是一起被访问的，我们实现了一种全有或全无的逐出策略，即要么逐出一个序列的所有块，要么一个都不逐出。此外，一个请求中的多个序列（例如，一个束搜索请求中的束候选）作为一个序列组被联合调度。由于这些序列之间可能存在内存共享，同一序列组内的序列始终被一起抢占或重新调度。为回

答第二个问题——如何恢复被逐出的块，我们考虑两种技术：

交换。这是大多数虚拟内存实现中使用的经典技术，将逐出的页面复制到磁盘上的交换空间。在我们的场景中，我们将逐出的块复制到 CPU 内存。如图 4 所示，除了 GPU 块分配器之外，vLLM 还包含一个 CPU 块分配器来管理交换到 CPU RAM 的物理块。当 vLLM 耗尽新 token 的空闲物理块时，它选择一组序列进行抢占，并将它们的 KV 缓存传输到 CPU。一旦抢占了一个序列并逐出了其块，vLLM 停止接受新请求，直到所有被抢占的序列完成为止。一旦某个请求完成，其块从内存中释放，被抢占序列的块被重新加载以继续处理该序列。注意，在此设计下，交换到 CPU RAM 的块数量永远不会超过 GPU RAM 中物理块的总数，因此 CPU RAM 上的交换空间受限于分配给 KV 缓存的 GPU 内存大小。

重新计算。在这种情况下，当被抢占的序列被重新调度时，我们简单地重新计算 KV 缓存。注意，重新计算的延迟可能显著低于原始延迟，因为在解码过程中生成的 token 可以与原始用户提示连接为一个新提示——它们所有位置的 KV 缓存可以在一个提示阶段迭代中生成。

交换和重新计算的性能取决于 CPU RAM 与 GPU 内存之间的带宽以及 GPU 的计算能力。我们在 §7.3 中考察了交换和重新计算的速度。

4.6 分布式执行

许多 LLM 的参数规模超出单个 GPU 的容量 [5, 9]。因此，有必要将其划分到分布式 GPU 上，并以模型并行方式执行 [28, 63]。这需要一个能处理分布式内存的内存管理器。vLLM 通过支持广泛使用的 Megatron-LM 式 Transformer 张量模型并行策略 [47]，在分布式环境下同样有效。该策略遵循 SPMD（单程序多数据）执行方案，其中线性层被划分以执行逐块矩阵乘法，GPU 通过 all-reduce 操作持续同步中间结果。具体而言，注意力算子在注意力头维度上被拆分，每个 SPMD 进程负责多头注意力中一部分注意力头。

我们观察到，即使在模型并行执行中，每个模型分片仍然处理相同的输入 token 集合，因此需要对相同位置使用 KV 缓存。因此，vLLM 在集中式调度器内设置了单一的 KV 缓存管理器，如图 4 所示。不同 GPU 工作节点共享该管理器，以及从逻辑块到物理块的映射。这种公共映射允许 GPU 工作节点根据调度器为每个输入请求提供的物理块来执行模型。尽管每个 GPU 工作节点具有相同的物理块 ID，但工作节点仅针对其对应的注意力头存储一部分 KV 缓存。

在每个步骤中，调度器首先准备消息，包含批次中每个请求的输入 token ID，以及每个请求的块表。接下来，调度器将此控制消息广播给 GPU 工作节点。然后，GPU 工作节点开始使用输入 token ID 执行模型。在注意力层中，GPU 工作节点根据控制消息中的块表读取 KV 缓存。在执行过程中，GPU 工作节点通过 all-reduce 通信原语同步中间结果，无需调度器的协调，如同 [47]。最后，GPU 工作节点将本次迭代的采样 token 发送回调度器。总之，GPU 工作节点无需在内存管理上同步，因为

Table 1. 模型大小和服务配置。

模型大小	13B	66B	175B
GPU	A100	4×A100	8×A100-80GB
总 GPU 内存	40 GB	160 GB	640 GB
参数大小	26 GB	132 GB	346 GB
KV 缓存内存	12 GB	21 GB	264 GB
最大 KV 缓存槽位数	15.7K	9.7K	60.1K

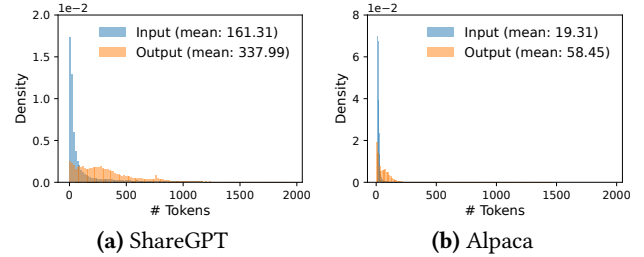


Figure 11. (a) ShareGPT 和 (b) Alpaca 数据集的输入和输出长度分布。

它们只需在每次解码迭代开始时随步骤输入一起接收所有内存管理信息。

5 实现

vLLM 是一个端到端推理服务系统，具有 FastAPI [15] 前端和基于 GPU 的推理引擎。前端扩展了 OpenAI API [34] 接口，允许用户为每个请求自定义采样参数，如最大序列长度和束宽度 k 。vLLM 引擎使用 8.5K 行 Python 和 2K 行 C++/CUDA 代码编写。我们使用 Python 开发控制相关组件，包括调度器和块管理器，同时为 PagedAttention 等关键操作开发自定义 CUDA kernel。对于模型执行器，我们使用 PyTorch [39] 和 Transformers [58] 实现了主流 LLM，如 GPT [5]、OPT [62] 和 LLaMA [52]。我们使用 NCCL [32] 进行跨分布式 GPU 工作节点的张量通信。

5.1 Kernel 级优化

由于 PagedAttention 引入了现有系统无法高效支持的内存访问模式，我们开发了多个 GPU kernel 来优化它。(1) 融合重塑和块写入。在每一层 Transformer 中，新的 KV 缓存被分割成块，重塑为针对块读取优化的内存布局，然后根据块表保存到指定位置。为了最小化 kernel 启动开销，我们将它们融合到一个 kernel 中。(2) 融合块读取和注意力。我们改编了 FasterTransformer [31] 中的注意力 kernel，使其根据块表读取 KV 缓存并即时执行注意力操作。为确保合并内存访问，我们分配一个 GPU warp 来读取每个块。此外，我们增加了对请求批次内可变序列长度的支持。(3) 融合块复制。由写时复制机制触发的块复制操作可能操作非连续的块。如果使用 cudaMemcpyAsync API，这可能导致大量小数据移动的调用。为降低开销，我们实现了一个 kernel，将不同块的复制操作批量合并到单个 kernel 启动中。

5.2 支持各种解码算法

vLLM 使用三个关键方法实现各种解码算法：fork、append 和 free。fork 方法从现有序列创建一个新序列。append

方法向序列追加一个新 token。最后，free 方法删除序列。例如，在并行采样中，vLLM 使用 fork 方法从单个输入序列创建多个输出序列。然后在每次迭代中使用 append 向这些序列添加新 token，并使用 free 删除满足停止条件的序列。同样的策略也应用于 vLLM 中的束搜索和前缀共享。我们相信未来的解码算法也可以通过组合这些方法得到支持。

6 评估

在本节中，我们在多种工作负载下评估 vLLM 的性能。

6.1 实验设置

模型与服务器配置。我们使用参数规模为 13B、66B 和 175B 的 OPT [62] 模型，以及参数规模为 13B 的 LLaMA [52] 模型进行评估。13B 和 66B 是大语言模型 (LLM) 的热门参数规模，正如某 LLM 排行榜 [38] 所示，而 175B 则是著名的 GPT-3 [5] 模型的参数规模。在所有实验中，我们使用 Google Cloud Platform 上搭载 NVIDIA A100 GPU 的 A2 实例。详细的模型规模和服务器配置见表 1。

工作负载。我们基于 ShareGPT [51] 和 Alpaca [50] 数据集合成了工作负载，这些数据集包含真实 LLM 服务的输入和输出文本。ShareGPT 数据集是用户分享的与 ChatGPT [35] 对话的集合。Alpaca 数据集是由 GPT-3.5 通过 self-instruct [57] 方法生成的指令数据集。我们对数据集进行 token 化处理，并使用其输入和输出长度来合成客户端请求。如图 11 所示，ShareGPT 数据集的平均输入提示比 Alpaca 数据集长 8.4 $\times$ ，输出长 5.8 $\times$ ，且方差更大。由于这些数据集不包含时间戳，我们使用泊松分布以不同的请求到达率生成请求到达时间。

基线 1: FasterTransformer。FasterTransformer [31] 是一个高度针对延迟优化的分布式推理引擎。由于 FasterTransformer 没有自己的调度器，我们实现了一个自定义调度器，其动态批处理机制类似于现有的服务系统 (如 Triton [30])。具体来说，根据 GPU 内存容量，我们在每个实验中设置尽可能大的最大批次大小 B 。调度器最多取 B 个最早到达的请求，将整个批次发送给 FasterTransformer 进行处理。

基线 2: Orca。Orca [60] 是一个针对吞吐量优化的先进 LLM 服务系统。由于 Orca 尚未公开可用，我们实现了自己的 Orca 版本。我们假设 Orca 使用伙伴分配算法来确定存储 KV 缓存的内存地址。我们根据 Orca 对请求输出空间的预留程度，实现了三个版本的 Orca：

- **Orca (Oracle)。**我们假设系统已知请求实际将生成的输出长度。这展示了 Orca 的性能上界，但在实践中无法实现。
- **Orca (Pow2)。**我们假设系统对输出空间的预留最多为 2 $\times$ 。例如，若实际输出长度为 25，则预留 32 个位置的输出空间。
- **Orca (Max)。**我们假设系统始终预留至模型最大序列长度的空间，即 2048 个 token。

关键指标。我们关注服务吞吐量。具体来说，使用不同请求到达率的工作负载，我们测量系统的归一化延迟，

即每个请求的端到端延迟除以其输出长度的均值，与 Orca [60] 一致。一个高吞吐量的服务系统应在高请求到达率下保持较低的归一化延迟。在大多数实验中，我们使用 1 小时的请求轨迹评估系统。作为例外，受成本限制，我们对 OPT-175B 模型使用 15 分钟的请求轨迹。

6.2 基本采样

我们在三个模型和两个数据集上评估 vLLM 在基本采样 (每个请求一个样本) 下的性能。图 12 的第一行展示了在 ShareGPT 数据集上的结果。曲线表明，随着请求到达率增加，延迟最初以渐进速度增长，但随后会突然飙升。这可以归因于当请求到达率超过服务系统的容量时，队列长度会持续无限增长，请求延迟也随之无限增长。

在 ShareGPT 数据集上，vLLM 能够比 Orca (Oracle) 承受 1.7 $\times$ -2.7 $\times$ 更高的请求到达率，比 Orca (Max) 承受 2.7 $\times$ -8 $\times$ 更高的请求到达率，同时保持相似的延迟。这是因为 vLLM 的 PagedAttention 可以高效管理内存使用，从而比 Orca 批处理更多的请求。例如，如图 13a 所示，对于 OPT-13B，vLLM 同时处理的请求数是 Orca (Oracle) 的 2.2 $\times$ ，是 Orca (Max) 的 4.3 $\times$ 。与 FasterTransformer 相比，vLLM 可以承受高达 22 $\times$ 的请求到达率，因为 FasterTransformer 未使用细粒度的调度机制，且与 Orca (Max) 一样低效地管理内存。

图 12 的第二行和图 13b 展示了在 Alpaca 数据集上的结果，其趋势与 ShareGPT 数据集相似。一个例外是图 12 (f)，其中 vLLM 相对于 Orca (Oracle) 和 Orca (Pow2) 的优势不那么明显。这是因为 OPT-175B 的模型和服务器配置 (表 1) 提供了充足可用的 GPU 内存空间来存储 KV 缓存，而 Alpaca 数据集的序列较短。在这种配置下，尽管 Orca (Oracle) 和 Orca (Pow2) 的内存管理存在低效之处，它们仍然可以批处理大量请求。结果，系统性能变为计算受限而非内存受限。

6.3 并行采样与束搜索

我们通过两种常用的采样方法——并行采样和束搜索，评估 PagedAttention 中内存共享的有效性。在并行采样中，一个请求内的所有并行序列可以共享提示的 KV 缓存。如图 14 的第一行所示，当采样序列数越多时，vLLM 相对于 Orca 基线的改进越大。类似地，图 14 的第二行展示了不同束宽下束搜索的结果。由于束搜索允许更多的共享，vLLM 展现出更大的性能优势。在 OPT-13B 和 Alpaca 数据集上，vLLM 相对于 Orca (Oracle) 的改进从基本采样中的 1.3 $\times$ 提升到束宽为 6 的束搜索中的 2.3 $\times$ 。

图 15 绘制了内存节省量，计算方法为通过共享节省的块数除以未共享时的总块数。我们展示了并行采样下 6.1% - 9.8% 的内存节省，以及束搜索下 37.6% - 55.2% 的内存节省。在 ShareGPT 数据集的相同实验中，我们观察到并行采样下 16.2% - 30.5% 的内存节省，以及束搜索下 44.3% - 66.3% 的内存节省。

6.4 共享前缀

我们探索了 vLLM 在不同输入提示共享同一前缀情况下的有效性，如图 10 所示。我们使用多语言模型 LLaMA-13B [52]。对于工作负载，我们使用 WMT16 [4] 英德翻译

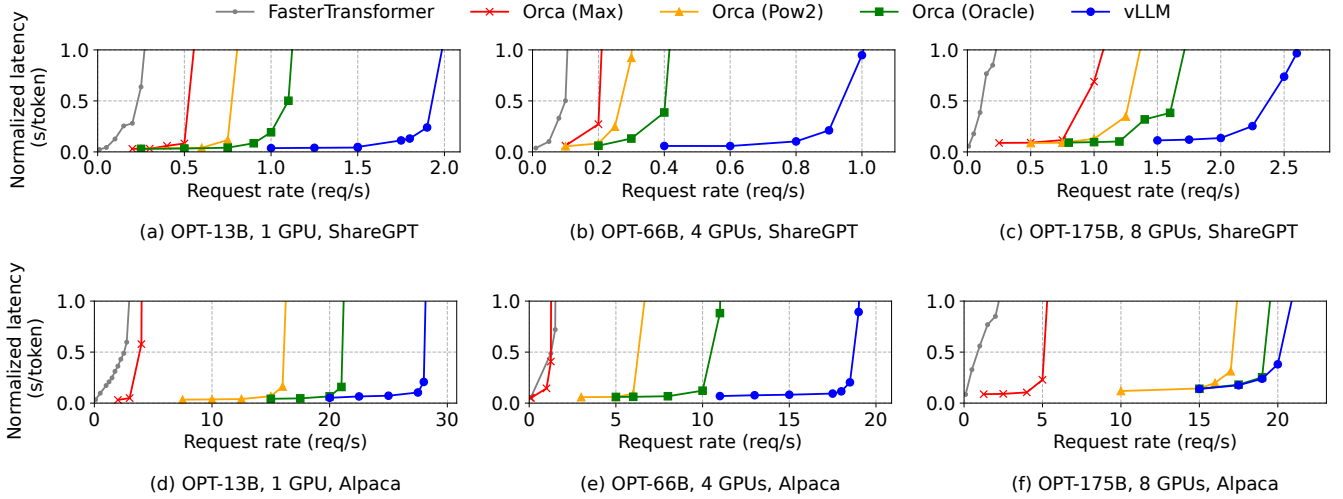


Figure 12. 在 ShareGPT 和 Alpaca 数据集上使用 OPT 模型进行单序列生成

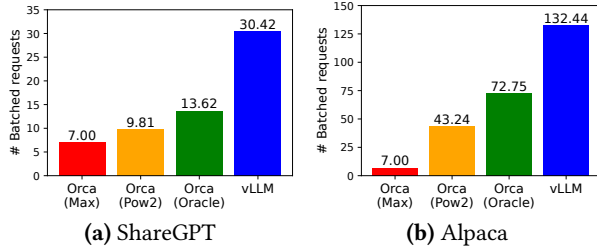


Figure 13. 在 ShareGPT (2 请求/秒) 和 Alpaca (30 请求/秒) 的请求轨迹下, OPT-13B 服务的平均批处理请求数。

数据集, 并合成了两个前缀, 分别包含一条指令和若干翻译示例。第一个前缀包含一个示例 (即 one-shot), 另一个前缀包含 5 个示例 (即 few-shot)。如图 16 (a) 所示, 当共享 one-shot 前缀时, vLLM 的吞吐量比 Orca (Oracle) 高 1.67 $\times$ 。此外, 当共享更多示例时 (图 16 (b)), vLLM 的吞吐量比 Orca (Oracle) 高 3.58 $\times$ 。

6.5 聊天机器人

聊天机器人 [8, 19, 35] 是 LLM 最重要的应用之一。为实现一个聊天机器人, 我们通过将聊天历史和最后一条用户查询拼接成一个提示, 让模型生成回复。我们使用 ShareGPT 数据集合成聊天历史和用户查询。由于 OPT-13B 模型的上下文长度有限, 我们将提示截断至最后 1024 个 token, 并让模型最多生成 1024 个 token。我们不在不同的对话轮次之间存储 KV 缓存, 因为这样会在对话轮次之间占用本可供其他请求使用的空间。

图 17 显示, vLLM 能够比三个 Orca 基线承受 2 $\times$ 更高的请求到达率。由于 ShareGPT 数据集包含许多长对话, 大多数请求的输入提示有 1024 个 token。由于伙伴分配算法, Orca 基线为请求输出预留了 1024 个 token 的空间, 无论它们如何预测输出长度。因此, 三个 Orca 基线的表现相似。相比之下, vLLM 可以有效处理长提示, 因为 PagedAttention 解决了内存碎片化和预留的问题。

7 消融研究

在本节中, 我们研究 vLLM 的各个方面, 并通过消融实验评估我们所做的设计选择。

7.1 Kernel 微基准测试

PagedAttention 中的动态块映射会影响涉及存储 KV 缓存的 GPU 操作性能, 即块读/写和注意力。与现有系统相比, 我们的 GPU kernel (§5) 涉及访问块表、执行额外分支以及处理可变序列长度的额外开销。如图 18a 所示, 与高度优化的 FasterTransformer 实现相比, 这导致注意力 kernel 延迟增加 20–26%。我们认为这一开销较小, 因为它仅影响注意力算子而非模型中的其他算子 (如 Linear)。尽管存在这一开销, PagedAttention 使 vLLM 在端到端性能上显著优于 FasterTransformer (§6)。

7.2 块大小的影响

块大小的选择会对 vLLM 的性能产生显著影响。如果块大小太小, vLLM 可能无法充分利用 GPU 的并行性来读取和处理 KV 缓存。如果块大小太大, 内部碎片增加且共享概率降低。

在图 18b 中, 我们使用 ShareGPT 和 Alpaca 数据集在固定请求率下的基本采样, 评估了不同块大小下 vLLM 的性能。在 ShareGPT 数据集上, 块大小从 16 到 128 可获得最佳性能。在 Alpaca 数据集上, 虽然块大小 16 和 32 表现良好, 但更大的块大小显著降低性能, 因为序列变得比块大小更短。在实践中, 我们发现块大小 16 足够大以高效利用 GPU, 又足够小以避免大多数工作负载中的显著内部碎片。因此, vLLM 将其默认块大小设为 16。

7.3 重新计算与交换的对比

vLLM 同时支持重新计算和交换作为其恢复机制。为理解这两种方法之间的权衡, 我们评估了它们的端到端性能并微基准测试了它们的开销, 结果如图 19 所示。我们的结果表明, 在小块大小下交换会导致过高的开销。这是因为小块大小通常导致 CPU 和 GPU 之间大量小数据传输, 限制了有效 PCIe 带宽。相反, 重新计算的开销

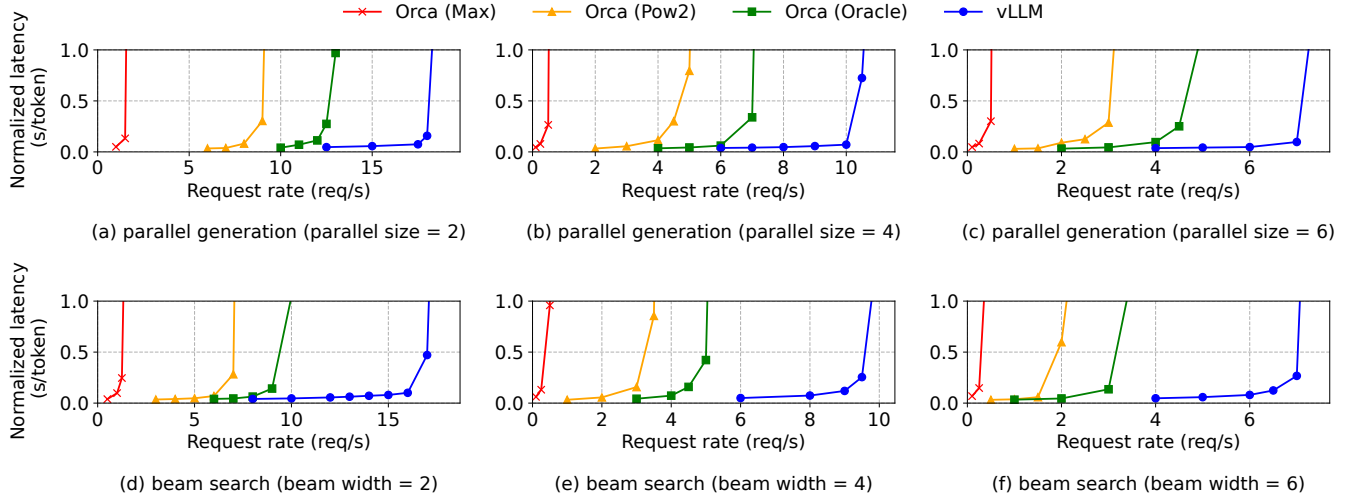


Figure 14. 在 Alpaca 数据集上使用 OPT-13B 进行并行生成和束搜索。

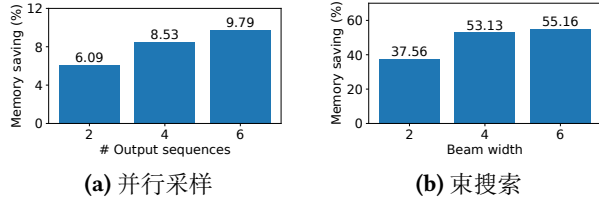


Figure 15. 在 Alpaca 请求轨迹下，OPT-13B 服务时通过共享 KV 块节省的平均内存量。

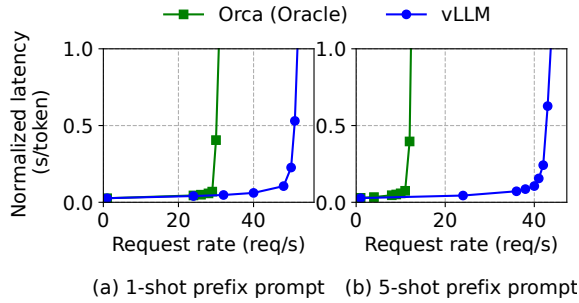


Figure 16. 输入提示共享同一前缀的翻译工作负载。前缀包含 (a) 1 个示例 (80 个 token) 或 (b) 5 个示例 (341 个 token)。

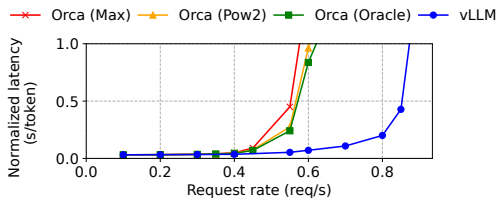


Figure 17. 聊天机器人工作负载下的性能。

在不同块大小下保持不变，因为重新计算不使用 KV 块。因此，当块大小较小时，重新计算更高效；当块大小较

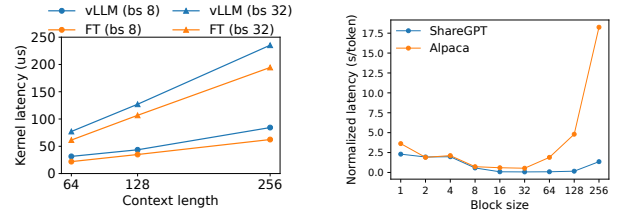


Figure 18. (a) 注意力 kernel 的延迟。 (b) 不同块大小下的端到端延迟。

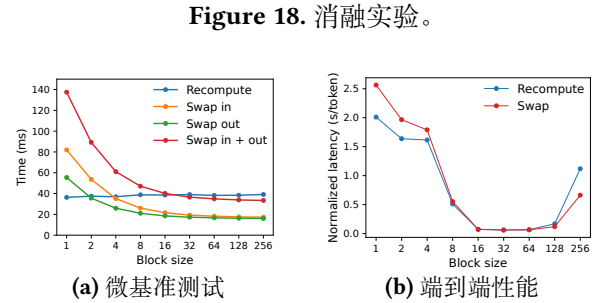


Figure 19. (a) 不同块大小下重新计算和交换的开销。 (b) 在相同请求率下使用 ShareGPT 数据集服务 OPT-13B 时的性能。

大时，交换更高效，尽管重新计算的开销从不高于交换延迟的 20%。对于 16 到 64 的中等块大小，两种方法展现出相当的端到端性能。

8 讨论

将虚拟内存和分页技术应用于其他 GPU 工作负载虚拟内存和分页的思想对大语言模型 (LLM) 推理服务中的 KV 缓存管理十分有效，因为该工作负载需要动态内存分配（输出长度事先未知），且其性能受限于 GPU 内存容量。然而，这并非普遍适用于所有 GPU 工作负载。例如，在 DNN 训练中，张量形状通常是静态的，因此内

存分配可以提前优化。又如，在非大语言模型 (LLM) 的 DNN 推理服务中，提升内存效率可能不会带来任何性能改善，因为性能主要受限于计算能力。在此类场景下，引入 vLLM 的技术反而可能因内存间接访问和非连续块内存带来的额外开销而导致性能下降。尽管如此，我们期待看到 vLLM 的技术被应用于与大语言模型 (LLM) 推理服务具有相似特性的其他工作负载。

在应用虚拟内存和分页时针对大语言模型 (LLM) 的特定优化 vLLM 通过利用应用特定的语义，重新诠释并增强了虚拟内存和分页的思想。一个例子是 vLLM 的全有或全无换出策略，该策略利用了处理一个请求需要其所有对应的 token 状态都存储在 GPU 内存中这一事实。另一个例子是恢复被驱逐块的重计算方法，这在操作系统中是不可行的。此外，vLLM 通过将用于内存访问操作的 GPU kernel 与用于注意力等其他操作的 kernel 进行融合，减轻了分页中内存间接访问的开销。

9 相关工作

通用模型推理服务系统近年来，模型推理服务一直是活跃的研究领域，涌现出大量针对深度学习模型部署各个方面的系统。Clipper [11]、TensorFlow Serving [33]、Nexus [45]、InferLine [10] 以及 Clockwork [20] 是一些较早的通用模型推理服务系统。它们研究了针对单个或多个模型推理服务的批处理、缓存、放置和调度。最近，DVABatch [12] 引入了多入口多出口批处理。REEF [21] 和 Shepherd [61] 提出了推理服务中的抢占机制。AlpaServe [28] 利用模型并行实现统计复用。然而，这些通用系统未能考虑大语言模型 (LLM) 推理的自回归特性和 token 状态，导致错失了优化机会。

面向 Transformer 的专用推理服务系统由于 Transformer 架构的重要性，涌现了众多面向它的专用推理服务系统。这些系统利用 GPU kernel 优化 [1, 29, 31, 56]、先进的批处理机制 [14, 60]、模型并行 [1, 41, 60] 以及参数共享 [64] 来实现高效推理服务。其中，Orca [60] 与我们的方法最相关。

与 Orca 的比较 Orca [60] 中的迭代级调度与 vLLM 中的 PagedAttention 是互补的技术：两个系统都旨在提高 GPU 利用率，从而提升大语言模型 (LLM) 推理服务的吞吐量，但 Orca 通过对请求进行调度和交织处理，使得更多请求可以并行处理，而 vLLM 则通过提高内存利用率，使得更多请求的工作集能够装入内存。通过减少内存碎片并支持共享，vLLM 在一个批次中并行运行更多请求，相比 Orca 实现了 2-4× 的加速。事实上，类似 Orca 中那样细粒度的请求调度和交织处理使内存管理变得更加困难，这使得 vLLM 提出的技术更加至关重要。

内存优化加速器计算能力与内存容量之间日益扩大的差距，使内存成为训练和推理的瓶颈。交换 [23, 42, 55]、重计算 [7, 24] 及其组合 [40] 已被用于降低训练的内存峰值。值得注意的是，FlexGen [46] 研究了在有限 GPU 内存下如何交换权重和 token 状态以进行大语言模型 (LLM) 推理，但它并不针对在线推理服务场景。OLLA [48] 优化了张量的生命周期和位置以减少碎片，但它没有进行

细粒度的块级管理或在线推理服务。FlashAttention [13] 应用分块和 kernel 优化来降低注意力计算的内存峰值并减少 I/O 开销。本文在在线推理服务的背景下引入了块级内存管理的新思想。

10 结论

本文提出了 PagedAttention，一种允许注意力键值对存储在非连续分页内存中的新型注意力算法，并介绍了 vLLM，一个由 PagedAttention 实现高效内存管理的高吞吐量语言模型 (LLM) 推理服务系统。受操作系统启发，我们展示了如何将虚拟内存和写时复制等成熟技术应用于大语言模型 (LLM) 推理服务中，以高效管理 KV 缓存并处理各种解码算法。我们的实验表明，vLLM 相比当前最先进的系统实现了 2-4× 的吞吐量提升。

致谢

我们感谢 Xiaoxuan Liu、Zhifeng Chen、Yanping Huang、匿名 SOSP 审稿人以及我们的 shepherd Lidong Zhou 提供的宝贵反馈。本研究部分得到了来自 Andreessen Horowitz、Anyscale、Astronomer、Google、IBM、Intel、Lacework、Microsoft、Mohamed Bin Zayed University of Artificial Intelligence、Samsung SDS、Uber 和 VMware 的捐赠支持。

References

- [1] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, et al. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. *arXiv preprint arXiv:2207.00032* (2022).
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).
- [3] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A neural probabilistic language model. *Advances in neural information processing systems* 13 (2000).
- [4] Ondrej Bojar, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, Varvara Logacheva, Christof Monz, Matteo Negri, Aurelie Neveol, Mariana Neves, Martin Popel, Matt Post, Raphael Rubino, Carolina Scarton, Lucia Specia, Marco Turchi, Karin Verspoor, and Marcos Zampieri. 2016. Findings of the 2016 Conference on Machine Translation. In *Proceedings of the First Conference on Machine Translation*. Association for Computational Linguistics, Berlin, Germany, 131–198. <http://www.aclweb.org/anthology/W/W16/W16-2301>
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [7] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [8] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. <https://lmsys.org/blog/2023-03-30-vicuna/>

- [9] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311* (2022).
- [10] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. 2020. InferLine: latency-aware provisioning and scaling for prediction serving pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing*. 477–491.
- [11] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 613–627.
- [12] Weihao Cui, Han Zhao, Quan Chen, Hao Wei, Zirui Li, Deze Zeng, Chao Li, and Minyi Guo. 2022. DVABatch: Diversity-aware Multi-Entry Multi-Exit Batching for Efficient Processing of DNN Services on GPUs. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 183–198.
- [13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [14] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. 2021. TurboTransformers: an efficient GPU serving system for transformer models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 389–402.
- [15] FastAPI. 2023. FastAPI. <https://github.com/tiangolo/fastapi>.
- [16] Pin Gao, Lingfan Yu, Yongwei Wu, and Jinyang Li. 2018. Low latency rnn inference with cellular batching. In *Proceedings of the Thirteenth EuroSys Conference*. 1–15.
- [17] Amir Gholami, Zhewei Yao, Sehoon Kim, Michael W Mahoney, and Kurt Keutzer. 2021. Ai and memory wall. *RiseLab Medium Post* 1 (2021), 6.
- [18] Github. 2022. <https://github.com/features/copilot>
- [19] Google. 2023. <https://bard.google.com/>
- [20] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving {DNNs} like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 443–462.
- [21] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale Preemption for Concurrent {GPU-accelerated} {DNN} Inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 539–558.
- [22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [23] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1341–1355.
- [24] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. 2020. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *Proceedings of Machine Learning and Systems* 2 (2020), 497–511.
- [25] Tom Kilburn, David BG Edwards, Michael J Lanigan, and Frank H Sumner. 1962. One-level storage system. *IRE Transactions on Electronic Computers* 2 (1962), 223–235.
- [26] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691* (2021).
- [27] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190* (2021).
- [28] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. *arXiv preprint arXiv:2302.11665* (2023).
- [29] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling holistic deep learning compiler optimizations with rtasks. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. 881–897.
- [30] NVIDIA. [n.d.]. Triton Inference Server. <https://developer.nvidia.com/nvidia-triton-inference-server>.
- [31] NVIDIA. 2023. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>.
- [32] NVIDIA. 2023. NCCL: The NVIDIA Collective Communication Library. <https://developer.nvidia.com/nccl>.
- [33] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv preprint arXiv:1712.06139* (2017).
- [34] OpenAI. 2020. <https://openai.com/blog/openai-api>
- [35] OpenAI. 2022. <https://openai.com/blog/chatgpt>
- [36] OpenAI. 2023. <https://openai.com/blog/custom-instructions-for-chatgpt>
- [37] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774 [cs.CL]*
- [38] LMSYS ORG. 2023. Chatbot Arena Leaderboard Week 8: Introducing MT-Bench and Vicuna-33B. <https://lmsys.org/blog/2023-06-22-leaderboard/>.
- [39] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [40] Shishir G Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph Gonzalez. 2022. POET: Training Neural Networks on Tiny Devices with Integrated Rematerialization and Paging. In *International Conference on Machine Learning*. PMLR, 17573–17583.
- [41] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2022. Efficiently Scaling Transformer Inference. *arXiv preprint arXiv:2211.05102* (2022).
- [42] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training.. In *USENIX Annual Technical Conference*. 551–564.
- [43] Reuters. 2023. <https://www.reuters.com/technology/tech-giants-ai-like-bing-bard-poses-billion-dollar-search-problem-2023-02-22/>
- [44] Amazon Web Services. 2023. <https://aws.amazon.com/bedrock/>
- [45] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. Nexus: A GPU cluster engine for accelerating DNN-based video analysis. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 322–337.
- [46] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E Gonzalez, et al. 2023. High-throughput Generative Inference of Large Language Models with a Single GPU. *arXiv preprint arXiv:2303.06865* (2023).
- [47] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

- [48] Benoit Steiner, Mostafa Elhoushi, Jacob Kahn, and James Hegarty. 2022. OLLA: Optimizing the Lifetime and Location of Arrays to Reduce the Memory Usage of Neural Networks. (2022). <https://doi.org/10.48550/arXiv.2210.12924>
- [49] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. *Advances in neural information processing systems* 27 (2014).
- [50] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
- [51] ShareGPT Team. 2023. <https://sharegpt.com/>
- [52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [54] Jing Wang, Youyou Lu, Qing Wang, Minhui Xie, Keji Huang, and Jiwu Shu. 2022. Pacman: An Efficient Compaction Approach for {Log-Structured} {Key-Value} Store on Persistent Memory. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 773–788.
- [55] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming*. 41–53.
- [56] Xiaohui Wang, Ying Xiong, Yang Wei, Mingxuan Wang, and Lei Li. 2021. LightSeq: A High Performance Inference Library for Transformers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers*. 113–120.
- [57] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-Instruct: Aligning Language Model with Self Generated Instructions. *arXiv preprint arXiv:2212.10560* (2022).
- [58] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
- [59] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144* (2016).
- [60] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for {Transformer-Based} Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [61] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: Serving DNNs in the Wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 787–808. <https://www.usenix.org/conference/nsdi23/presentation/zhang-hong>
- [62] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).
- [63] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter-and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 559–578.
- [64] Zhe Zhou, Xuechao Wei, Jieqing Zhang, and Guangyu Sun. 2022. PetS: A Unified Framework for Parameter-Efficient Transformers Serving. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 489–504.